

Задача 4. Алгоритм RSA

Имя входных файлов: задается в параметрах командной строки

Имя выходного файла: стандартный вывод

Ограничения по времени: -

Ограничения по памяти: -

Часть 1.

Реализовать следующие алгоритмы работы с числами **(1)** и алгоритмы, реализующие арифметику сравнений:

1. Add – сложение двух чисел по модулю N.
2. Multiply – умножение двух чисел по модулю N.
3. ModExp – возведение в степень по модулю N.
4. Euclid – алгоритм Эвклида.
5. ExtendedEuclid – расширенный алгоритм Эвклида.
6. Primality – проверка числа на простоту.
7. Primality2 – проверка числа на простоту.
8. GetRandomPrim – получение случайного простого числа.
9. Алгоритм RSA.

В качестве базового типа взять *unsigned char*.

Часть 2.

Реализовать алгоритмы **Части 1**, но работающие с длинными числами (n).

Величину *n* выбрать в зависимости от номера варианта:

Вариант	(n)	Базовый тип
1	2	unsigned short int
2	4	unsigned short int
3	8	unsigned short int
4	16	unsigned short int
5	2	unsigned long int
6	4	unsigned long int
7	8	unsigned long int
8	16	unsigned long int
9	2	unsigned char
10	4	unsigned char
11	8	unsigned char

Формат входных данных

Для алгоритма *Add*:

```
<project> mul <mod> <num1> <num2>
```

Для алгоритма *Multiply*:

```
<project> mul <mod> <num1> <num2>
```

Для алгоритма *ModExp*:

```
<project> mod <mod> <num1> <num2>
```

Для алгоритма *Euclid, ExtendedEuclid*:

```
<project> euc <mod> <num1> <num2>
```

```
<project> eucx <mod> <num1> <num2>
```

Для алгоритма *Primality, Primality2*:

```
<project> prim <num>
```

```
<project> prim2 <num>
```

Для алгоритма *GetRandomPrim*:

```
<project> rnd <size>
```

Для алгоритма *RSA*:

```
<project> rsa -getkeys
```

```
<project> rsa code <publickey> <text>
```

```
<project> rsa decode <secretkey> <text>
```

Формат выходных данных

Пусть

x – значение, записанное в `<num1>`;

y – значение, записанное в `<num2>`,

N – значение, переданное параметром `<mod>`.

Для алгоритма **Add**: значение $x+y \bmod N$

Для алгоритма **Multiply**: значение $x*y \bmod N$

Для алгоритма **ModExp**: значение $x^y \bmod N$

Для алгоритма **Euclid**: $\text{НОД}(x, y)$

Для алгоритма **ExtendedEuclid**: пара чисел, возвращаемых алгоритмом Extended Euclid

Для алгоритма **Primality, Primality2**: 1 – если число простое, 0 – если составное.

Для алгоритма **GetRandomPrim**: случайное число.

Для алгоритма **RSA**: необходимо продемонстрировать пошаговую работу алгоритма с выводом промежуточных результатов, а также осуществить проверку правильности шифрования, проведя дешифровку полученного закодированного числа.